

A 计算dps

考察基础语法、如何将浮点数保留两位输出。

直接模拟即可。

注意输出格式，换行空格等不能错。

```
#include <bits/stdc++.h>
using namespace std;

void solve(){
    double s; int d;
    scanf("%lf%d", &s, &d);
    double v = d/s;
    if(v > 3000) printf("%.21f super big\n", v);
    if(v <= 3000 && v > 1000) printf("%.21f big\n", v);
    if(v <= 1000) printf("%.21f medium\n", v);
}

int main(){
    int t; scanf("%d", &t);
    while(t-->0) solve();
    return 0;
}
```

B 城市规划

如果能画出 $n=4$ 的解，基于这个解，很容易构造出任意 n 的解：

$n=1$ 、 2 、 3 时，无解，输出 -1

$n=4$ 时，一种可行的方案为：

```
4
1 2
2 1
2 4
3 4
```

此时 $k=1$ ，而关键街区至少要选 2 个。

$n>4$ 时，在上述方案的点 4 后面连出去一条链就行了

比如 $n=8$ ：

```
8
1 2
2 1
2 4
3 4
4 5
5 6
6 7
7 8
```

本质是将所有强连通分量缩成一个点，形成一张新图之后，新图上入度为 0 的点的个数就是关键街区的数量。

所以从大小至少为 2 的强连通分量为起点，构造一张满足要求的图。

C 小Z的银行

尽量选择小的 a_i ,可以满足更多人。考虑从小到大将 a_i 排序,

用任意一种时间复杂度为 $O(n\log n)$ 的排序算法进行排序。

用 $O(n^2)$ 的排序算法会超时。注意数据范围和边界, c语言选手要开long long, 数组要开够大小。

```
#include <bits/stdc++.h>
#define ll long long
using namespace std;
const int N = 200010;

ll n, m, a[N];
int main(){
    scanf("%lld%lld", &n, &m);
    for(int i = 0; i < n; i++)
        scanf("%lld", &a[i]);
    sort(a, a+n);
    int cur = 0;
    for(int i = 0; i < n; i++)
        if(m >= a[i]) cur++, m -= a[i];
    printf("%d\n", cur);
}
```

D 吹风机和洗衣机

一个符合直觉的想法是：尽量把洗衣机放在角落，排成一个尽量方的方形更优。假设洗衣机排在一个 $p \times q$ 的长方形范围内，比如，如果洗衣机有10个，可以被安排在 3×4 的方形范围内。

具体来说，有四种情况：

1、吹风机为0或洗衣机为0的情况。

2、 $p = n$ 的情况

3、 $q = m$ 的情况

4、 $p < n$ 且 $q < m$ 的情况。此时答案为 $n * m - x - p - q - 1$ ，也就是说，我们要找到一对 p, q ，满足 $p * q > x$ ，使得 $p + q$ 最大。

由数学知识， p, q 最多差1，差2及以上就可以在和不变的情况下对 p, q 调整，使得 $p * q$ 更大。

可以在 $\sqrt{x}$ 附近讨论或直接计算 p, q 的值和对应的答案。

```
#include<bits/stdc++.h>
#pragma GCC optimize(2)
#define ll long long
#define fi first
#define se second
using namespace std;
typedef pair<int,int> pii;
const int maxn=2e5+10;
ll inf=1007LL*1007*1007;
ll n,m,x;
void qmax(ll &x,ll y) { x=max(x,y); }
void solve(){
    cin>>n>>m>>x;
    if(x==0){
        cout<<n*m<<"\n";
        return;
    }
    if(x==n*m){
        cout<<"0\n";
        return;
    }
    ll ans=0;
    if(x%n==0){
        qmax(ans,n*m-x-n);
    }
    else qmax(ans,n*m-x-n-1);
    if(x%m==0){
        qmax(ans,n*m-x-m);
    }
    else qmax(ans,n*m-x-m-1);
    ll L=0,R=min({n,m,inf})+1,mx=0;
    while(L+1<R){
        ll mid=(L+R)>>1;
        if(mid*mid>=x){
            R=mid;
            mx=mid;
        }
    }
```

```

        else L=mid;
    }
    if(mx) qmax(ans,n*m-mx-mx-x-1);
    L=0,R=min({n,m,inf}),mx=0;
    while(L+1<R){
        ll mid=(L+R)>>1;
        if(mid*(mid+1)>=x){
            R=mid;
            mx=mid;
        }
        else L=mid;
    }
    if(mx) qmax(ans,n*m-mx-(mx+1)-x-1);
    cout<<ans<<"\n";
}

signed main(){
    int t; cin>>t;
    while(t--) solve();
    return 0;
}

```

E 密文版静待宣告

样例2解释：

```
6
1 2 3 3 2 1
```

六天获得的密文版种类分别是，1，2，3，3，2，1

一种可行的方案是：

第三天获得密文版后，我们手上有1，2，3，我们将2和3进行宣告，手上仅剩下密文版 1

第四天获得密文版后，手上有1，3，我们将这两个密文版宣告。

第六天获得密文版后，手上有2，1，我们将这两个密文版宣告。

所以是Yes。

对于没看懂题的同学致歉，应该加一个样例解释。

解法：

对每个 a_i ，记录下一次出现相同种类密文版的位置为 $next_i$ 。

进行判断：如果现在不做操作的话，下一天是否会让手上的密文版坍塌。如果是，选择next最小的两个密文版进行宣告。

如果无法进行上述操作，答案为No，如果上述操作能顺利进行，输出Yes

按上述操作，最后手上可能还会剩下一些密文版。可以证明这些密文版经过调整，能在之前的某些天宣告。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

void solve()
{
    int n, v=0;
    cin >> n;
    vector<int> a(n+1);
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
        v = max(v, a[i]);
    }
    vector<int> nxt(n+1);
    vector<int> pos(v+1, n+1);
    for (int i = n; i; --i) {
        nxt[i] = pos[a[i]];
        pos[a[i]] = i;
    }

    int op = 0;
    vector<int> cnt(v+1);
    priority_queue<pair<int,int>,vector<pair<int,int>>,greater<>> q;
    for (int i = 1; i <= n; ++i) {
        if (++cnt[a[i]] == 2) {
            cout << "No\n"; return;
        }
    }
}
```

```

    }
    q.emplace(nxt[i], a[i]);
    ++op;
    if (q.size() >= 2 && op>0 && q.top().first == i + 1) {
        int x = q.top().second; q.pop();
        int y = q.top().second; q.pop();
        --cnt[x];
        --cnt[y];
        --op;
    }
}
cout << "Yes\n";
}

int main()
{
    cin.tie(nullptr)->sync_with_stdio(false);
    int t = 1;
    cin >> t;
    while (t--) { solve(); }
}

```


F 该毕设了？该网瘾了！

经过数学计算发现，前面 $n-1$ 天全都网瘾，最后一天毕设最优。

注意特判 $n=1$ 的情况。

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long
string Solve()
{
    int n, m, x, p;
    cin >> n >> m >> x >> p;
    if(n==1&&m==1) return "Yes";
    ll now = 1;
    for (int i = 1; i < n; i++)
    {
        now = now * (1 + p);
        if (now >= m)
            return "Yes";
    }
    return "No";
}
int main()
{
    ios::sync_with_stdio(false), cin.tie(0);
    int t; cin>>t;
    while(t--)
        cout << Solve() << "\n";
    return 0;
}
```

G 小F吃谷

价格为 i 的谷子最多买 $\frac{m}{i}$ 个。我们对于每个价格 i ，处理出最大的 $\frac{m}{i}$ 个愉悦值。

由于 $\frac{m}{i}$ 形成一个调和级数，有效物品的个数是 $O(m \log m)$ 级别的。

剩下就是01背包的模板。总复杂度 $O(m^2 \log m)$

```
#include<bits/stdc++.h>
#pragma GCC optimize(2)
#define ll long long
#define fi first
#define se second
using namespace std;
typedef pair<int,int> pii;
const int maxn=2e5+10;
int inf=1007LL*1007*1007;
int n,m;
int a[maxn],b[maxn],c[maxn];
multiset <pii,greater<pii>> s[1005];
int ans[1005];
void solve(){
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        cin>>a[i];
    }
    for(int i=1;i<=n;i++){
        cin>>b[i];
    }
    for(int i=1;i<=n;i++){
        cin>>c[i];
    }
    for(int i=1;i<=n;i++){
        s[a[i]].insert({b[i],c[i]});
    }
    vector <pii> ver;
    for(int i=1;i<=m;i++){
        if(!s[i].size()) continue;
        for(int j=1;j<=m/i;j++){
            auto it=s[i].begin();
            pii nw=*it; nw.fi-=nw.se;
            ver.push_back(({*it).fi,i});
            s[i].erase(it);
            s[i].insert(nw);
        }
    }
    memset(ans,-0x3f,sizeof(ans));
    ans[0]=0;
    for(auto [v,w]:ver){
        for(int j=m;j>=w;j--){
            ans[j]=max(ans[j],ans[j-w]+v);
        }
    }
    cout<<*max_element(ans,ans+m+1)<<"\n";
}
```

```
signed main(){  
    int t; t=1;  
    while(t-->0) solve();  
    return 0;  
}
```

H 小树的课表

考察模拟能力，涉及字符串处理。直接按题意模拟即可。

I 随机序列

用程序1生成的序列大概长这个样子：

```
202 528 614 676 808 949 959 974 1014 1113 1146 1218 1394 1396 1428 1516 1558 1569 1665 1668 1764
1923 2053 2123 2234 2360 2709 2808 2845 2948 2973 3094 3470 3620 3625 3700 3732 3744 3768 3959
4105 4115 4381 4567 4616 4618 4633 4829 4913 5121 5142 5157 5288 5395 5516 5590 5617 5635 5729
5932 5932 6025 6083 6142 6185 6186 6300 6303 6328 6353 6487 6757 7006 7031 7346 7541 7550 7691
7737 7858 7893 7927 7951 8058 8293 8310 8520 8606 8935 9037 9200 9320 9323 9480 9552 9722 9728
9876 9880 9940
```

用程序2生成的序列大概长这个样子：

```
4249 5662 5979 7089 9069 9310 9942 9948 9987 9987 9997 9999 9999 10000 10000 10000 10000 10000
10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000
10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000
10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000
10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000
10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000 10000
10000 10000
```

可以看到明显的差别，程序2生成的序列，最后会出现很多连续的10000，并且整个序列的 平均值/总和 会显著更大。

出题人的本意是，自己写一个随机程序，或者凭感觉分析，找到一个总和的分界点就能通过。发现最后会出现连续很多个10000的同学也能通过。

还有其他基于两个序列的特点的做法也能通过。

我们写代码计算过第95个数为10000的概率。保留小数点后10位的结果是1.0000000000，可能有浮点误差，但足够说明问题。

我们随机生成了一百万个序列，发现所有程序2生成的序列，总和都大于9000*100

这道题也有严谨的做法，我们可以用数学公式计算出，对于一个给定序列，是程序1生成的概率有多少，是程序2生成的概率有多少。用python或高精度可以实现，但这种方法不是出题人本意。

已知 $a_1 \sim a_{100}$,

是程序1生成的概率为：

$$\frac{n!}{10000^{100}}$$

$$\text{程序2: } \frac{1}{10000} \times \prod_{k=1}^{99} \frac{1}{10001-a_k}$$

可以通分后比较大小。

J 串串

使用字符串哈希，预处理出T的每个前缀和后缀。

对于每个S，暴力枚举分割点，在之前预处理出的前缀和后缀哈希值里查找，判断是否合法。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef unsigned long long ull;
#define pll pair<ll, ll>
#define pii pair<int, int>
#define ls first
#define rs second
#define endl '\n'
#define FAST ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
#define pb push_back
#define eb emplace_back
// #define ll int
const int N = 1e6 + 3;
const ll mod1 = 1e9 + 7;
const ll mod2 = 998244353;
const double eps = 1e-8;
const double inf = 1e20;
const ll INF = 1e18;
ll gcd(ll a, ll b) { return b == 0 ? a : gcd(b, a % b); }
ll lcm(ll a, ll b) { return a / gcd(a, b) * b; }
ll floordiv(ll a, ll b) { return (a % b && a <= 0) ? a / b - 1 : a / b; }
ll ceildiv(ll a, ll b) { return (a % b && a > 0) ? a / b + 1 : a / b; }
ll n, m;
ll hash1[N], hash2[N], pow1[N], pow2[N];
unordered_map<ll, pii> pre, suf;
ll calc(ll x, ll y)
{
    return (x << 31) + y;
}
ll get_hash(ll l, ll r)
{
    ll res1 = 0, res2 = 0;
    res1 = ((hash1[r] - hash1[l - 1] * pow1[r - l + 1] % mod1) % mod1 + mod1) % mod1;
    res2 = ((hash2[r] - hash2[l - 1] * pow2[r - l + 1] % mod2) % mod2 + mod2) % mod2;
    return calc(res1, res2);
}
void solve()
{
    // cout<<bitset<64>(1000000000 + 6)<<endl;
    // cout<<bitset<64>(calc(1,1000000000 + 6))<<endl;
    pow1[0] = 1;
    pow2[0] = 1;
    for (int i = 1; i <= 100000; i++)
    {
        pow1[i] = pow1[i - 1] * mod2 % mod1;
        pow2[i] = pow2[i - 1] * mod1 % mod2;
    }
    cin >> n >> m;
```

```

for (int i = 1; i <= n; i++)
{
    string s;
    cin >> s;
    int len = s.length();
    s = " " + s;
    for (int j = 1; j <= len; j++)
    {
        hash1[j] = (hash1[j - 1] * mod2 % mod1 + s[j]) % mod1;
        hash2[j] = (hash2[j - 1] * mod1 % mod2 + s[j]) % mod2;
    }
    for (int j = 1; j <= len; j++)
    {
        ll pp = calc(hash1[j], hash2[j]);
        ll ss = get_hash(j, len);
        pre[pp].ls++;
        pre[pp].rs = i;
        suf[ss].ls++;
        suf[ss].rs = i;
    }
}
while (m--)
{
    string s;
    cin >> s;
    int len = s.length();
    s = " " + s;
    for (int j = 1; j <= len; j++)
    {
        hash1[j] = (hash1[j - 1] * mod2 % mod1 + s[j]) % mod1;
        hash2[j] = (hash2[j - 1] * mod1 % mod2 + s[j]) % mod2;
    }
    bool flag = 0;
    for (int j = 1; j < len; j++)
    {
        ll pp = calc(hash1[j], hash2[j]);
        ll ss = get_hash(j + 1, len);
        // if((pre.find(pp)!=pre.end())&&(suf.find(ss)!=suf.end())&&
        (pre[pp].ls>1||suf[ss].ls>1||pre[pp].rs!=suf[ss].rs))
        // {
        //     flag=1;
        //     cout<<j<<" ";
        //     cout<<"Yes"<<endl;
        //     break;
        // }
        if ((pre.find(ss) != pre.end()) && (suf.find(pp) != suf.end()) && (pre[ss].ls > 1 ||
suf[pp].ls > 1 || pre[ss].rs != suf[pp].rs))
        {
            flag = 1;
            // cout<<j<<" ";
            cout << "Yes" << endl;
            break;
        }
    }
}
if (!flag)
{

```

```
        cout << "No" << endl;
    }
}
return;
}
int main()
{
    FAST;
    ll t = 1;
    // cin >> t;
    while (t--)
    {
        solve();
    }
    return 0;
}
```